

ICT600 CORRECTED STUDENT EDITION

July 2024 Corrected Student Scheme

Corrected categories, constraints, PHP access control, and secure form handling

Scope	July 2024 final examination
Evidence	Printed question, supplied scheme where available, current lecture notes, and technical verification
Status	100 marks: Part A 70, Part B 30

Prepared 26 September 2026

How to use this reference

Teaching-use document

Attempt the original question before revealing this scheme. This corrected edition is not an official UiTM replacement; it records known source errors and accepts equivalent technically correct answers.

- Assessment total: 100 marks: Part A 70, Part B 30.
- Model answers show one defensible solution; alternative correct wording or code may receive full credit.
- For code, method marks remain available when the approach is sound and a minor syntax error does not invalidate the solution.
- Security improvements are shown where historical examples trusted or displayed unvalidated input.

Q1

HTML applications and five web-based CMS tools.

[18 marks]

Model answer

- HTML applications: structure documents, hyperlink/embed resources, create forms, and include multimedia (2 each).
- CMS: WordPress, Blogger, Joomla, Drupal, and Weebly/Wix or another accepted course example (2 each).

Marking guide: Q1(a) 8; Q1(b) 10.

Q2(a)

Describe six website-anatomy components.

[6 marks]

Model answer

- Containing block, logo, navigation, content, footer, and whitespace, each with its function.

Marking guide: 1 mark per correct component and brief purpose.

Q2(b)

Name six client-side development tools.

[6 marks]

Model answer

- Official categories are JavaScript libraries, front-end frameworks, web-application frameworks, programming languages, IDEs, and web-application testing tools.
- Accept named tools when the relevant category is clear, for example jQuery (library), Bootstrap (front-end framework), VS Code/NetBeans (IDE), and a browser testing tool.

Marking guide: 1 mark per valid category or unambiguous category-example pair.

Correction / moderation note

The generated scheme required specific products, while the official answer expects tool categories. The corrected rubric accepts both without confusing categories and products.

Q3(a)

Classify four mechanisms that enforce database restrictions.

[8 marks]

Model answer

- Primary key, foreign key, UNIQUE constraint, and NOT NULL or CHECK constraint, each with a correct effect.
- A plain non-unique index improves lookup performance but does not itself restrict data; accept only a unique index when restriction is claimed.

Marking guide: Name (1) + accurate restriction (1) for each of four.

Correction / moderation note

The official and generated schemes treat a plain index as a restriction mechanism. This is technically inaccurate.

Q3(b)

Insert the three ITEM records.

[10 marks]

Model answer

- Use INSERT INTO ITEM with the correct column list and three VALUES rows; quote text and keep numeric values numeric.

Marking guide: Table/columns (2), VALUES (2), three complete records (6).

Q4

PHP multidimensional car array and string-trace output.

[10 marks]

Model answer

- Store and display Proton, Perodua, and Toyota stock/sold rows (4).
- Output: 34; original; uppercase; lowercase; News->Story result; substring; int(21); bool(false) (6).

Marking guide: Q4(a) 4; Q4(b) 6.

Q5

Restrict user_view.php to administrators and implement logout.

[12 marks]

Model answer

- Start/read the session, reject non-administrators, redirect/include an error page, and stop protected processing (6).
- Detect logout, clear session data, destroy the session, redirect, and stop processing (6).
- For production systems, perform logout with POST plus CSRF protection; the historical question explicitly supplies a GET hyperlink.

Marking guide: 6 marks per subpart.

Part B Q1 - corrected patient-name analyser

Study the complete working exemplar below.

[See printed allocation]

Model answer

- Equivalent correct implementations that satisfy the stated behaviour should receive credit.

```
<!-- inputData.html -->
<form method="post" action="receiveData.php">
  <label>DATA: <textarea name="data" required></textarea></label>
  <button type="submit">SEND</button>
</form>
```

```

<!-- receiveData.php -->
<?php
$raw = trim($_POST['data'] ?? '');
$names = array_values(array_filter(array_map('trim', explode(',', $raw))));
$dataArray = [];
foreach ($names as $name) {
    $dataArray[$name] = mb_strlen($name);
}
arsort($dataArray);
$longestName = array_key_first($dataArray);
$longestLength = $longestName === null ? 0 : $dataArray[$longestName];

function e(string $v): string { return htmlspecialchars($v, ENT_QUOTES, 'UTF-8'); }
?>
<p>Longest name: <?= e($longestName ?? 'No name supplied') ?></p>
<p>Length: <?= $longestLength ?></p>

```

Marking guide: Apply the original question's subpart allocation and award method marks consistently.

Correction / moderation note

The generated answer assumes POST data exists and silently overwrites duplicate names. This version validates input and safely encodes the displayed name. Part B Q2 should use the same secure registration pattern as January 2024.

Part B Q2 - self-contained secure registration exemplar

Create the required registration form and PHP handling.

[15 marks]

Model answer

- This student edition includes the complete secure pattern instead of referring to another semester.

```

<?php
$values = ['fullname' => '', 'email' => '', 'birthdate' => '', 'gender' => ''];
$message = '';

function e(string $value): string {
    return htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    foreach ($values as $key => $unused) {
        $values[$key] = trim($_POST[$key] ?? '');
    }
    $password = $_POST['password'] ?? '';
    if ($values['fullname'] === '' || $values['email'] === '' || $password === '') {
        $message = 'Unsuccessful: enter all required information.';
    } else {
        $message = 'Successful signup for ' . $values['fullname'] . ' .';
    }
}

?>
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>User Signup</title></head>
<body>
<h1>User Signup</h1>
<form method="post">
    <label>Full name* <input name="fullname" value="<?= e($values['fullname']) ?>" required</label><br>
    <label>Email* <input type="email" name="email" value="<?= e($values['email']) ?>" required</label><br>
    <label>Password* <input type="password" name="password" required</label><br>
    <label>Birthdate <input type="date" name="birthdate" value="<?= e($values['birthdate']) ?>"</label><br>
    <label><input type="radio" name="gender" value="M"> Male</label>
    <label><input type="radio" name="gender" value="F"> Female</label><br>
    <button type="submit">Signup</button>
</form>
<?php if ($message !== ''): ?><p><?= e($message) ?></p><?php endif; ?>
</body></html>

```

Marking guide: Form and controls (6), POST/validation (5), success/error output (2), safe output and structure (2).

Correction / moderation note

The earlier compact scheme cross-referenced January 2024. This edition is self-contained.

Consistency rules

- Award equivalent technically correct syntax and wording.
- Do not reproduce a known historical error merely because it appears in a supplied scheme.
- Keep printed subpart totals fixed unless a documented moderation decision says otherwise.
- For scenario questions, connect the explanation or mitigation to the scenario.
- Record additional accepted alternatives for the next controlled revision.